Optimal Route Planning for Orienteering Races on Real Terrain: An Integrated Cost Surface and Branch-and-Cut Approach

Pablo Borrego Ramos*

April 13, 2026

Abstract

We present an end-to-end computational framework for solving the Orienteering Problem (OP) on real terrain, combining a terrain-aware asymmetric cost model with exact combinatorial optimisation via Branch-and-Cut. The cost surface combines three layers: (i) an Areal Cost Raster (ACR) derived from IOF orienteering map symbols, (ii) a Hypsometry Cost Raster (HCR) capturing terrain morphometry via the Terrain Ruggedness Index, plan curvature, and slope magnitude, and (iii) a directional metabolic cost model based on the Minetti polynomial. A cumulative fatigue model modulates travel cost as a function of elapsed effort. The resulting asymmetric cost matrix feeds a Branch-and-Cut (B&C) solver that provides provably optimal solutions, warm-started by a Simulated Annealing (SA) metaheuristic. We validate the framework on two Spanish terrains—Torremocha and La Muela—across seven control-point distributions each. The B&C certifies global optimality on 10 of 14 instances within a 15-minute time limit.

Keywords: orienteering problem, least-cost path, branch-and-cut, terrain cost surface, Minetti metabolic model, simulated annealing

1 Introduction

The Orienteering Problem (OP) asks a competitor to select a subset of control points and determine a route starting and ending at a depot, maximising the total score collected while respecting a travel budget. Formally introduced by Tsiligirides [1984], the OP is NP-hard and has been studied extensively in operations research. However, most formulations assume symmetric, Euclidean distances, which poorly approximate real-world orienteering where terrain, vegetation, slope direction, and fatigue create highly asymmetric travel costs.

In competitive orienteering and rogaine events, route choice is the decisive skill: the runner must balance point value against travel difficulty across heterogeneous terrain. Course setters estimate ideal times using subjective experience, but the underlying cost function is a nonlinear function of terrain type, slope, and accumulated fatigue.

This paper bridges two research streams: (i) GIS-based cost surface modelling for orienteering maps, following Albert and Sárközy [2021], and (ii) exact optimisation of the OP via integer programming. Our contributions are:

1. A terrain cost model combining IOF map symbols (ACR), DEM-derived morphometry (HCR), directional metabolic cost (Minetti), and cumulative fatigue into a single asymmetric cost matrix.
2. An IQR-based scaling method for merging ACR and HCR that adapts to terrain characteristics without manual calibration.

*Independent Researcher.

3. A Branch-and-Cut formulation for the asymmetric OP with fatigue, including McCormick linearisation of the bilinear fatigue-time product.
4. Computational experiments on two real terrains with seven control distributions, demonstrating provable optimality on multiple instances.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the cost surface model. Section 4 presents the mathematical formulation. Section 5 details the solution methods. Section 6 outlines the computational pipeline. Section 7 describes the experimental setup, and section 8 presents the results, including a synthetic benchmark suite for the asymmetric OP with fatigue. Section 9 discusses findings and limitations, and section 10 concludes.

2 Related Work

The Orienteering Problem. The OP was introduced by Tsiligirides [1984] and has since generated a substantial body of literature. Vansteenwegen et al. [2011] provide a comprehensive survey covering exact methods, heuristics, and variants including the Team OP, OP with Time Windows, and stochastic OP. Exact approaches typically employ branch-and-bound or branch-and-cut on integer programming formulations [Fischetti et al., 1998, Gendreau et al., 1998]. Most formulations assume symmetric distances derived from Euclidean or road-network metrics. Gunawan et al. [2016] extend the survey to recent variants and applications, noting that terrain-aware formulations remain scarce.

Metaheuristics for the OP. Given the NP-hardness of the OP, metaheuristics are widely used for practical instances. Simulated Annealing [Kirkpatrick et al., 1983], Genetic Algorithms [Tasgetiren, 2002], and Ant Colony Optimisation [Ke et al., 2008] have all been applied. Vansteenwegen et al. [2009] propose an iterated local search that is competitive with exact methods on benchmark instances. Our SA implementation incorporates Or-opt moves and stagnation detection, with parameters tuned via grid search over 1,000 random instances.

Terrain cost modelling. Least-cost path (LCP) analysis on terrain has a long history in GIS. Tobler [1993] proposed the hiking function relating slope to walking speed, which remains widely used despite known limitations on steep descents. Minetti et al. [2002] measured the metabolic cost of locomotion across a wide range of slopes, producing a fifth-degree polynomial that better captures the energetics of both ascent and descent. Rees [2004] compared several slope-speed models and found significant differences in predicted travel times.

Orienteering map cost surfaces. Albert and Sárközy [2021] introduced a GIS-based approach to route planning on orienteering maps, assigning travel speeds to IOF symbol categories and computing least-cost paths. They proposed the Hypsometry Cost Raster (HCR) combining terrain ruggedness, plan curvature, and slope magnitude to capture morphometric difficulty not represented by map symbols. Our work extends their approach by: (i) replacing Tobler's hiking function with the Minetti metabolic model for directional slope costs, (ii) proposing IQR-based scaling instead of histogram mode matching for ACR-HCR fusion, and (iii) coupling the cost surface with an exact OP solver rather than point-to-point LCP analysis.

3 Cost Surface Model

Let the terrain be discretised into a regular grid of cells with resolution δ (metres per cell), where each cell is indexed by its row r and column c. Each cell (r, c) is assigned a base traversal cost that integrates map-derived and DEM-derived information.

3.1 Areal Cost Raster (ACR)

The ACR assigns a dimensionless weight $w_{rc} \in (0, 10]$ to each cell based on the IOF map symbol classification. Following Albert and Sárközy [2021], each symbol category maps to a running speed, and the weight is inversely proportional:

$$w_{rc} = \frac{\delta}{v_{\text{base}} \cdot s_{\text{cat}(r,c)}} \tag{1}$$

where $v_{\text{base}} = 2.5\,\text{m/s}$ is the reference flat-terrain speed, s_{cat} is the speed factor for the symbol category (e.g., $s = 1.0$ for paved road, $s = 0.133$ for fight vegetation), and δ is the cell size. Impassable features (water, cliffs, buildings) receive $w = 10$.

The ACR is rasterised from the orienteering map file (OMAP format) by parsing symbol objects—lines, areas, and points—and rendering them onto the UTM grid with appropriate buffer widths. Bézier curves are tessellated to preserve cartographic accuracy.

3.2 Hypsometry Cost Raster (HCR)

The HCR captures terrain morphometry not represented by map symbols, derived from a Digital Terrain Model (DTM). Following Albert and Sárközy [2021], three morphometric variables are computed and normalised to $[\ell, u] = [0.1, 1.0]$:

Terrain Ruggedness Index (TRI). The standard deviation of elevation in a 3×3 neighbourhood [Riley et al., 1999]:

$$\text{TRI}_{rc} = \sqrt{\frac{1}{9} \sum_{(i,j) \in \mathcal{N}(r,c)} \left(z_{ij} - \bar{z}_{rc}\right)^2} \tag{2}$$

where z_{ij} is the elevation at cell (i, j) and $\bar{z}_{rc}$ is the local mean.

Plan Curvature (PC). The absolute value of plan curvature detects ridges, spurs, and side valleys [Zevenbergen and Thorne, 1987]:

$$\text{PC}_{rc} = \left| \frac{-\left(p^2 r - 2pqs + q^2 t\right)}{\left(p^2 + q^2\right)^{3/2}} \right| \tag{3}$$

where $p = \partial z/\partial x$, $q = \partial z/\partial y$, $r = \partial^2 z/\partial x^2$, $s = \partial^2 z/\partial x \partial y$, $t = \partial^2 z/\partial y^2$, computed via central finite differences at resolution δ.

Slope Magnitude (SLO). The slope angle in degrees:

$$\text{SLO}_{rc} = \arctan \sqrt{\left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2} \tag{4}$$

Each variable is normalised via robust percentile clipping:

$$\hat{V}_{rc} = \ell + (u - \ell) \cdot \frac{\text{clip}(V_{rc}, V^{(1\%)}, V^{(99\%)}) - V^{(1\%)}}{V^{(99\%)} - V^{(1\%)}} \tag{5}$$

The HCR is then:

$$\text{HCR}_{rc} = \widehat{\text{TRI}}_{rc}^{\,\alpha} \cdot \left|\widehat{\text{PC}}_{rc}\right|^{\beta} \cdot \widehat{\text{SLO}}_{rc}^{\,\gamma} \tag{6}$$

with $\alpha = 1$, $\beta = 0.5$, $\gamma = 1$ as specified by Albert and Sárközy [2021].

3.3 IQR-Based Raster Fusion

To combine ACR and HCR, the HCR must be scaled into ACR's numerical space. Albert and Sárközy [2021] proposed histogram mode matching: $c = f_{\text{ACR}}(\mu)/f_{\text{HCR}}(\mu)$, where $f(\mu)$ denotes the density at the mode. This is sensitive to distribution shape and can produce unstable scaling constants. For example, when the ACR distribution is multimodal (due to discrete symbol categories) and the HCR distribution is unimodal and right-skewed, the mode densities $f(\mu)$ can differ by an order of magnitude, yielding scaling constants that vary erratically with small changes in bin width.

We evaluated six scaling methods on both study areas: median ratio, mean ratio, IQR ratio, standard deviation ratio, percentile range matching, and full quantile mapping. The median and mean ratios produced excessively large constants ($c > 7$) due to the different distribution shapes of ACR (discrete symbol weights) and HCR (continuous, right-skewed). Percentile and quantile mapping over-corrected by forcing HCR to mimic ACR's distribution. The IQR ratio was selected for its robustness to outliers and its focus on matching variability rather than central tendency, adapting automatically to terrain characteristics ($c = 0.87$ for Torremocha, $c = 2.18$ for La Muela). A straightforward alternative, normalising both rasters to $[0, 1]$, was rejected because it would discard the ACR's physically meaningful scale, where cell weights directly encode running speed ratios derived from IOF symbol specifications.

We propose an Interquartile Range (IQR) ratio:

$$c = \frac{\text{IQR}(\text{ACR})}{\text{IQR}(\text{HCR})} = \frac{Q_{75}^{\text{ACR}} - Q_{25}^{\text{ACR}}}{Q_{75}^{\text{HCR}} - Q_{25}^{\text{HCR}}} \tag{7}$$

where Q_p denotes the p-th percentile over valid cells. This matches the *variability* of HCR to ACR rather than aligning central tendencies, producing a more stable and interpretable scaling.

The scaled HCR is added to ACR with path protection:

$$\text{Cost}_{rc} = w_{rc} + c \cdot \text{HCR}_{rc} \cdot \pi_{rc} \tag{8}$$

where the path protection factor π_{rc} prevents HCR from distorting well-mapped features:

$$\pi_{rc} = \begin{cases} 0.1 & \text{if } w_{rc} < 0.15 \text{ (paths/roads)} \\ 0.3 & \text{if } 0.15 \leq w_{rc} < 0.25 \text{ (good tracks)} \\ 0.0 & \text{if } w_{rc} \geq 5.0 \text{ (impassable)} \\ 1.0 & \text{otherwise} \end{cases} \tag{9}$$

The path protection factor attenuates HCR on cells where the cartographer has already provided precise cost information. Paths and roads (ISOM 501–505) are mapped with high positional accuracy in orienteering maps; DEM-derived roughness at these locations reflects the surrounding terrain rather than the path surface. The attenuation values (0.1 for paths, 0.3 for tracks) were calibrated to allow minimal HCR influence on linear features while preserving full contribution on open terrain, where map symbols provide only coarse vegetation categories. Impassable features ($w \geq 5.0$) receive zero HCR contribution to prevent softening of hard barriers.

3.4 Directional Metabolic Cost (Minetti Model)

The metabolic cost of locomotion on a slope gradient i (rise/run, dimensionless) is modelled by the fifth-degree polynomial of Minetti et al. [2002]:

$$C(i) = 280.5\, i^5 - 58.7\, i^4 - 76.8\, i^3 + 51.9\, i^2 + 19.6\, i + 2.5 \quad [\text{J/kg/m}] \tag{10}$$

valid for $i \in [-0.45, +0.45]$. The cost multiplier relative to flat terrain is:

$$\phi(i) = \frac{\max\big(C(i),\, 0.5\big)}{C(0)} \tag{11}$$

where the floor at 0.5 prevents unrealistic negative costs on steep descents.

During anisotropic Dijkstra pathfinding, the directional slope between adjacent cells $(r_1, c_1) \to (r_2, c_2)$ is computed from the DEM-derived slope gradients. At each grid cell, the partial derivatives $\partial z/\partial x$ and $\partial z/\partial y$ are computed via central finite differences on the DEM. For a step between two adjacent cells, these derivatives are averaged to obtain a representative gradient at the midpoint:

$$\bar{g}_x = \tfrac{1}{2}\left(\left.\frac{\partial z}{\partial x}\right|_{(r_1,c_1)} + \left.\frac{\partial z}{\partial x}\right|_{(r_2,c_2)}\right), \quad \bar{g}_y = \tfrac{1}{2}\left(\left.\frac{\partial z}{\partial y}\right|_{(r_1,c_1)} + \left.\frac{\partial z}{\partial y}\right|_{(r_2,c_2)}\right) \tag{12}$$

Let $\Delta c = c_2 - c_1$ and $\Delta r = r_2 - r_1$ be the step direction. The signed slope gradient along the direction of travel is:

$$i_{12} = \frac{\bar{g}_x \cdot \Delta c + \bar{g}_y \cdot \Delta r}{\sqrt{\Delta c^2 + \Delta r^2}} \tag{13}$$

This projects the terrain gradient onto the movement vector, yielding a positive value for uphill travel and negative for downhill. The traversal cost of a single Dijkstra step from cell (r_1, c_1) to an adjacent cell (r_2, c_2) is:

$$d_{12} = \|\Delta\| \cdot \frac{\mathrm{Cost}_{r_1 c_1} + \mathrm{Cost}_{r_2 c_2}}{2} \cdot \phi(i_{12}) \tag{14}$$

where Cost_{rc} is the combined base cost at cell (r, c) from eq. (8), $\phi(i_{12})$ is the Minetti slope factor from eq. (11), and $\|\Delta\| = \sqrt{\Delta c^2 + \Delta r^2}$ is the Euclidean step length between adjacent cells ($\|\Delta\| = 1$ for cardinal moves, $\|\Delta\| = \sqrt{2}$ for diagonal moves in the 8-connected grid). The base cost is averaged between the source and destination cells to smooth transitions across terrain boundaries.

This produces an *asymmetric traversal cost: the step cost d_{12} from cell 1 to cell 2 differs from d_{21} in the reverse direction, because the Minetti factor satisfies $\phi(i) \neq \phi(-i)$ for any non-zero slope—ascending a gradient $i > 0$ is more expensive than descending it.*

3.5 Cumulative Fatigue Model

Athlete performance degrades over time. We model fatigue as a linear multiplier on travel cost:

$$f(t) = 1 + \lambda \cdot \frac{t}{B} \tag{15}$$

where t is the elapsed base travel time, B is the raw time budget, and $\lambda = 0.2$ is the fatigue rate. At the end of the race ($t = B$), the athlete is 20% slower than at the start.

The effective (fatigue-adjusted) cost of traversing arc (i, j) after elapsed time t_i is:

$$d_{ij}^f(t_i) = d_{ij} \cdot \left(1 + \lambda \cdot \frac{t_i}{B}\right) \tag{16}$$

The total fatigue-adjusted route cost for a route $\sigma = (0, v_1, v_2, \ldots, v_k, 0)$ is:

$$D^f(\sigma) = \sum_{\ell=0}^{k} d_{v_\ell, v_{\ell+1}} \cdot \left(1 + \lambda \cdot \frac{T_\ell}{B}\right) \tag{17}$$

where $T_\ell = \sum_{m=0}^{\ell-1} d_{v_m, v_{m+1}}$ is the cumulative base cost up to leg ℓ, and $v_0 = v_{k+1} = 0$ (depot).

3.6 Asymmetric Cost Matrix Construction

Given n control points plus the depot (node 0), the pairwise cost matrix $\mathbf{C} \in \mathbb{R}^{(n+1)\times(n+1)}$ is computed by running anisotropic Dijkstra from each node on the combined cost grid (eq. (8)) with Minetti slope factors (eq. (11)).

For each node $i \in V$, a single-source anisotropic Dijkstra is run on the (optionally downsampled) grid, starting from the grid cell corresponding to node i. The algorithm explores all 8-connected neighbours using a priority queue, computing the traversal cost to each adjacent cell via eq. (14). Because each step cost is directional (as described above), the shortest-path cost from node i to node j generally differs from the cost from j to i, yielding an asymmetric cost matrix.

The cost matrix entry C_{ij} is then the shortest-path distance from node i's grid cell to node j's grid cell in this directed cost field. Running $n+1$ independent Dijkstra computations (one per node) yields the full matrix. Cells with base cost ≥ 9.0 (impassable features) are treated as hard barriers with a fixed penalty cost, preventing routes from crossing cliffs, water, or out-of-bounds areas.

The grid is optionally downsampled by factor s, averaging the base cost in $s \times s$ blocks and rescaling node coordinates accordingly. This reduces computation time from $O(nHW\log(HW))$ to $O\big(n\frac{HW}{s^2}\log\frac{HW}{s^2}\big)$ at the cost of path resolution.

The resulting matrix satisfies $C_{ij} \neq C_{ji}$ in general, with asymmetry measured as:

$$\mathrm{A}= \frac{\mathrm{mean}_{i\neq j}\,|C_{ij}-C_{ji}|}{\mathrm{mean}_{i\neq j}\,C_{ij}} \times 100\% \tag{18}$$

4 Mathematical Formulation

We formulate the Asymmetric Orienteering Problem with Fatigue (AOPF) as a mixed-integer linear program.

4.1 Sets and Parameters

Let $V = \{0, 1, \ldots, n\}$ be the set of nodes, where node 0 is the depot.

- $p_i \geq 0$: score (points) collected at node i, with $p_0 = 0$.
- C_{ij}: base travel cost from i to j (asymmetric, from section 3.6).
- B: raw time budget.
- λ: fatigue rate.

4.2 Decision Variables

- $x_{ij} \in \{0, 1\}$: 1 if arc (i, j) is traversed.
- $y_i \in \{0, 1\}$: 1 if node i is visited.
- $t_i \in [0, B]$: arrival time (cumulative base cost) at node i.
- $w_{ij} \geq 0$: McCormick auxiliary variable approximating $t_i \cdot x_{ij}$.

4.3 Objective

Maximise total score:

$$\max \sum_{i \in V} p_i\, y_i \tag{19}$$

4.4 Constraints

Flow conservation. Each visited node has exactly one incoming and one outgoing arc:

$$\sum_{j \in V \setminus \{i\}} x_{ji} = y_i \qquad \forall\, i \in V \tag{20}$$

$$\sum_{j \in V \setminus \{i\}} x_{ij} = y_i \qquad \forall\, i \in V \tag{21}$$

Depot. The depot is always visited: $y_0 = 1$, $t_0 = 0$.

Time propagation (MTZ-type). We adopt Miller–Tucker–Zemlin (MTZ) constraints for subtour elimination and time tracking. While flow-based formulations (single-commodity or multi-commodity flow) are known to provide tighter LP relaxations for routing problems, MTZ was chosen because it directly provides arrival-time variables t_i at each node. These variables are essential for the fatigue model: the bilinear term $t_i \cdot x_{ij}$ in the fatigue budget (eq. (27)) requires explicit arrival times, which flow-based formulations do not provide without additional reconstruction. For each arc (i, j) with $j \neq 0$:

$$t_j \geq t_i + C_{ij} - M_{ij}(1 - x_{ij}) \tag{22}$$

where the big-M constant is tightened to:

$$M_{ij} = \max\big(B - C_{0i} - C_{j0},\ C_{ij}\big) \tag{23}$$

The value $B - C_{0i-C_{j0}}$ is the maximum time that could have elapsed at node i on any feasible route that still uses arc (i, j) and returns to the depot: the route must spend at least C_{0i} reaching i and C_{j0} returning from j, so $t_i \leq B - C_{0i} - C_{j0}$ (we also need $M_{ij} \geq C_{ij}$ to ensure the constraint is non-trivial when $x_{ij} = 0$). This is tighter than the standard MTZ constant $M = n$ and reduces the LP relaxation gap.

McCormick linearisation. The fatigue budget constraint involves the bilinear term $t_i \cdot x_{ij}$. We introduce $w_{ij} \approx t_i \cdot x_{ij}$ with McCormick envelope constraints. Let $\bar{w}_{ij} = \min(B - C_{ij} - C_{j0},\ B - C_{i0})$ be the tightest upper bound on w_{ij}:

$$w_{ij} \geq t_i - \bar{w}_{ij}(1 - x_{ij}) \tag{24}$$

$$w_{ij} \leq \bar{w}_{ij}\, x_{ij} \tag{25}$$

$$w_{ij} \leq t_i \tag{26}$$

Constraint (**??**eq:mc2 forces $w_{ij} = 0$ when $x_{ij} = 0$ (the arc is unused); and constraint (26) ensures w_{ij} never ex

Fatigue budget. The total fatigue-adjusted travel cost must not exceed the budget:

$$\sum_{(i,j) \in A} C_{ij}\, x_{ij} + \frac{\lambda}{B} \sum_{(i,j) \in A} C_{ij}\, w_{ij} \leq B \tag{27}$$

where $A = \{(i, j) : i \neq j,\ x_{ij} \text{ exists}\}$. The first sum is the base cost; the second sum linearises the fatigue penalty $\sum C_{ij} \cdot \lambda \cdot t_i / B$ via the McCormick variables.

Arc elimination. Arcs that are structurally infeasible are fixed to zero a priori. An arc (i, j) is eliminated if the base cost alone exceeds the budget:

$$x_{ij} = 0 \quad \text{if } C_{ij} + C_{j0} > B \;\text{ or }\; C_{0i} + C_{ij} + C_{j0} > B \tag{28}$$

Additionally, we apply fatigue-aware arc elimination. Even on the shortest possible path using arc (i, j)—the direct route $0 \to i \to j \to 0$—the fatigue-adjusted cost must not exceed the budget:

$$x_{ij} = 0 \quad \text{if } C_{0i} + C_{ij}\left(1 + \frac{\lambda\, C_{0i}}{B}\right) + C_{j0}\left(1 + \frac{\lambda\,(C_{0i} + C_{ij})}{B}\right) > B \tag{29}$$

This eliminates arcs that are feasible under base cost but infeasible once fatigue is accounted for, reducing the LP size without excluding any feasible solution.

4.5 Subtour Elimination and Connectivity Cuts

Because the cost matrix is asymmetric ($C_{ij} \neq C_{ji}$), the arc variables x_{ij} are directed: $x_{ij} = 1$ means the route traverses from i to j, and x_{ji} is a separate variable for the reverse direction. This requires *directed* subtour elimination constraints, as opposed to the undirected SECs used in symmetric formulations.

For a subset $S \subseteq V \setminus \{0\}$, the directed SEC is:

$$\sum_{i \in S} \sum_{j \in S} x_{ij} \leq |S| - 1 \tag{30}$$

Additionally, connectivity cuts ensure that every visited node can reach and be reached from the depot:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq y_k \qquad \forall\, k \in S \tag{31}$$

$$\sum_{i \notin S} \sum_{j \in S} x_{ij} \geq y_k \qquad \forall\, k \in S \tag{32}$$

A combined two-way cut further tightens the relaxation:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} + \sum_{i \notin S} \sum_{j \in S} x_{ij} \geq 2\, y_k \quad \text{for some } k \in S \tag{33}$$

Separation routines. Two separation procedures identify violated subsets S in the fractional LP solution:

Subtour detection (Kosaraju SCC). A directed graph is constructed from arcs with $x_{ij} > 0.5$. Kosaraju's algorithm identifies all strongly connected components (SCCs) in $O(|V| + |A|)$ time. Any SCC that does not contain the depot constitutes a violated subset: the nodes form a cycle disconnected from the depot, violating the requirement that all visited nodes lie on a single path from and to node 0. For each such SCC S, all four cut types (eqs. (30) to (33)) are added.

Depot-unreachable detection (reverse BFS). Even when no disconnected SCC exists, the fractional solution may contain visited nodes from which the depot is not reachable following directed arcs. A reverse BFS from the depot on the fractional graph (arcs with $x_{ij} > 0.5$) identifies all nodes that *can* reach the depot. Any visited node ($y_i > 0.5$) not in this reachable set is depot-unreachable and requires an outgoing connectivity cut toward the depot side. The set of all such nodes is collected into a single subset S, and the same four cut families are applied.

This two-phase separation is essential for the asymmetric OP: Kosaraju detects closed subtours on the *directed* fractional graph (standard connected-component algorithms would miss directionality), while the reverse BFS catches "dangling" paths that leave the depot but cannot return—a violation pattern that arises frequently with asymmetric costs where the cheapest path from i to j may not have a feasible return.

Lifted cover cuts on the budget knapsack. The fatigue budget constraint (eq. (27)) has the structure of a knapsack inequality: a weighted sum of binary arc variables bounded by B. When the LP relaxation fractionally activates many expensive arcs that collectively exceed the budget, subtour elimination cuts alone cannot tighten this bound. We add lifted cover inequalities: a *cover* $\mathcal{C}$ is a subset of arc variables whose base costs sum to more than B, yielding the valid inequality

$$\sum_{k \in \mathcal{C}} x_k \leq |\mathcal{C}| - 1 \tag{34}$$

Covers are found by greedily selecting arcs with the highest LP values until the budget is exceeded, then checking if the inequality is violated. Each cover is optionally strengthened via sequential lifting, computing the maximum coefficient for non-cover variables while maintaining validity. Up to 3 cover cuts are added per cut-loop iteration alongside the SECs.

5 Solution Methods

5.1 Simulated Annealing (SA)

Method selection. The SA metaheuristic was selected after a comparative study on 21 benchmark instances across varying sizes (n = 20–100), asymmetry levels, and fatigue rates. Four methods were evaluated with equal computational budget (2 seconds per method per instance, 10 independent seeds for stochastic methods): Greedy Insertion (deterministic), Genetic Algorithm (GA, population 50), Ant Colony Optimisation (ACO, 30 ants), and Simulated Annealing (SA).

Table 1: Heuristic comparison: mean gap below SA (%) across 21 benchmark instances (equal 2 s time budget, 10 seeds per stochastic method). All differences are statistically significant (Wilcoxon signed-rank, $p < 0.001$).

Method	Mean gap	Std	Min gap	Max gap
Greedy	23.0%	9.4%	3.7%	40.0%
GA	49.2%	11.0%	24.6%	71.9%
ACO	8.1%	4.9%	1.3%	24.1%
SA	—	—	—	—

SA significantly outperforms all three alternatives under equal time budgets (Wilcoxon signed-rank test, $n = 21$ instances, all $p < 0.001$; SA achieves a higher mean score than each competitor on all 21 instances). The GA performs worst (48.4% below SA on average), as crossover on directed routes frequently produces infeasible offspring requiring expensive repair, and the rugged fitness landscape under asymmetric costs degrades convergence. ACO is the closest competitor (5.7% below SA) but its pheromone mechanism weakens on asymmetric instances where edge quality is context-dependent on elapsed time and travel direction—the gap widens to 20.9% on the hardest instance. Greedy Insertion (21.0% below SA) is fast but myopic, committing to locally attractive nodes without ability to backtrack.

SA's advantage stems from its single-solution trajectory search: it evaluates each candidate move in full context (current route, elapsed time, fatigue state, directional costs) without needing to decompose the solution into reusable components. The Or-opt and swap moves are particularly effective on asymmetric instances because they directly evaluate the actual cost change of rearranging the visit sequence, naturally accounting for directionality.

Parameter tuning. The SA parameters were fine-tuned via grid search over a separate set of 1,000 synthetic instances with randomised control-point distributions across both study areas.

The tunable parameters and their search ranges were:

- Initial temperature $T_0 \in \{50, 100, 200, 500\}$
- Final temperature $T_{\text{end}} \in \{0.01, 0.1, 1.0\}$
- Number of iterations $N_{\text{iter}} \in \{20000, 40000, 80000, 120000\}$
- Number of restarts $N_{\text{restart}} \in \{4, 8, 12, 20\}$
- Move probabilities $(p_{\text{remove}}, p_{\text{insert}}, p_{\text{or-opt}}, p_{\text{swap}})$, tested in 10 configurations
- Or-opt segment length $L_{\text{seg}} \in \{1\text{–}2, 1\text{–}3, 1\text{–}4\}$
- Stagnation limit as fraction of N_{iter}: $\{1/3, 1/4, 1/5, 1/6\}$

The best configuration, used throughout the experiments, is: $T_0 = 100$, $T_{\text{end}} = 0.1$, move probabilities $(0.30, 0.30, 0.20, 0.20)$, $L_{\text{seg}} \in \{1, 2, 3\}$, and stagnation limit $N_{\text{iter}}/5$. The number of iterations and restarts are auto-calibrated based on instance size n:

$$N_{\text{iter}} = \text{clip}(2500n,\ 10000,\ 120000) \tag{35}$$

$$N_{\text{restart}} = \text{clip}(3000/n,\ 40,\ 200) \tag{36}$$

The rationale is as follows. The neighbourhood size grows with n: more nodes means more possible insertions, removals, and swaps per iteration, so the search space requires proportionally more iterations to explore. The linear scaling $2500n$ was determined empirically during grid search as the point where additional iterations yielded diminishing returns on solution quality. The lower bound of 10,000 ensures sufficient exploration even for very small instances, while the upper bound of 120,000 caps runtime.

Conversely, restarts scale inversely with n: for small instances ($n \leq 15$), the search space is compact and a single SA run converges quickly, so diversity across restarts (up to 20) is more valuable than depth within a single run. For large instances ($n \geq 40$), each restart is expensive and the search space is rich enough that a single long run explores effectively, so fewer restarts (minimum 4) suffice. The total computational effort $N_{\text{iter}} \times N_{\text{restart}}$ remains approximately constant across instance sizes, keeping the SA runtime predictable.

The cooling schedule is geometric: $T_k = T_0 \cdot \rho^k$ with decay rate $\rho = (T_{\text{end}}/T_0)^{1/N_{\text{iter}}}$.

Neighbourhood structure. Starting from a greedy insertion solution, the SA applies four neighbourhood moves with the tuned probabilities:

1. **Remove** ($p = 0.30$): delete a random node from the route.
2. **Insert** ($p = 0.30$): add an unvisited node at its best position (minimum cost increase, computed in $O(k)$ where k is the current route length).
3. **Or-opt** ($p = 0.20$): extract a segment of 1–3 nodes and reinsert at a different position.
4. **Swap** ($p = 0.20$): exchange a visited node for an unvisited one.

Moves that violate the fatigue budget ($D^f(\sigma') > B$) are rejected. The search terminates early if no improvement is found for $N_{\text{iter}}/5$ consecutive iterations (stagnation detection).

The full procedure, including the iterated multi-restart wrapper, is given in algorithm 1.

Algorithm 1 Iterated Simulated Annealing for AOPF

Require: Cost matrix $\mathbf{C}$, scores $\mathbf{p}$, budget B, fatigue rate λ
Ensure: Best route σ^*

$\sigma^* \leftarrow \emptyset$, $z^* \leftarrow 0$
Compute $N_{\text{iter}} \leftarrow \text{clip}(2500n, 10000, 120000)$
Compute $N_{\text{restart}} \leftarrow \text{clip}(3000/n, 40, 200)$
for $r = 1, \ldots, N_{\text{restart}}$ **do**
    $\sigma \leftarrow \textsc{GreedyInsertion}(\mathbf{C}, \mathbf{p}, B)$
    $z \leftarrow \sum_{i \in \sigma} p_i, \quad T \leftarrow T_0, \quad \rho \leftarrow (T_{\text{end}}/T_0)^{1/N_{\text{iter}}}, \quad s \leftarrow 0$
    for $k = 1, \ldots, N_{\text{iter}}$ **do**
        if $s \geq N_{\text{iter}}/5$ **then break** ▷ Stagnation
        end if
        $T \leftarrow T \cdot \rho$
        Draw $u \sim \text{Uniform}(0, 1)$
        if $u < 0.30$ **then**
            $\sigma' \leftarrow \textsc{Remove}(\sigma)$ ▷ Delete random node
        else if $u < 0.60$ **then**
            $\sigma' \leftarrow \textsc{Insert}(\sigma)$ ▷ Add node at best position
        else if $u < 0.80$ **then**
            $\sigma' \leftarrow \textsc{OrOpt}(\sigma)$ ▷ Relocate segment of 1–3 nodes
        else
            $\sigma' \leftarrow \textsc{Swap}(\sigma)$ ▷ Exchange visited ↔ unvisited
        end if
        if $D^f(\sigma') > B$ **then**
            $s \leftarrow s + 1$; **continue** ▷ Infeasible move
        end if
        $z' \leftarrow \sum_{i \in \sigma'} p_i, \quad \Delta \leftarrow z' - z$
        if $\Delta > 0$ **or** $\exp(\Delta/T) > \text{Uniform}(0, 1)$ **then**
            $\sigma \leftarrow \sigma', \quad z \leftarrow z'$
            if $z > z^*$ **then** $z^* \leftarrow z, \quad \sigma^* \leftarrow \sigma, \quad s \leftarrow 0$
            else $s \leftarrow s + 1$
            end if
        else $s \leftarrow s + 1$
        end if
    end for
end for
return σ^*

5.2 Branch-and-Cut (B&C)

The B&C solver uses the LP relaxation of the AOPF formulation (section 4), solved via the HiGHS dual simplex [Huangfu and Hall, 2018]. HiGHS was chosen as it is open-source, provides a C API suitable for embedding in a custom B&C loop, and its dual simplex implementation supports efficient warm-starting after cut addition. The algorithm proceeds as follows:

Algorithm 2 Branch-and-Cut for AOPF

```
Build root LP relaxation
Warm-start incumbent z* ← SA solution value
Push root node onto stack
while stack non-empty and time < T_max do
    Pop node, apply variable fixings
    for k = 1, ..., K_max do                          ▷ Cut loop, K_max = 20
        Solve LP relaxation
        if z̄ ≤ z* + ε then prune and continue
        end if
        Find violated SECs via Kosaraju SCC
        Find depot-unreachable nodes via reverse BFS
        if no violations found then break
        end if
        Add SECs and connectivity cuts
    end for
    if solution is integer-feasible then
        Extract route, validate fatigue feasibility
        if score > z* then update incumbent
        end if
    else
        Branch on most fractional y_i
        Push child nodes (DFS order)
    end if
end while
if stack empty then solution is provably optimal
```

Key implementation details:

- Branching is performed on the node-visit variables y_i rather than arc variables x_{ij}. Setting $y_i = 0$ removes node i from all future solutions in the subtree, eliminating $O(n)$ arc variables at once; setting $y_i = 1$ forces the node to be visited. By contrast, fixing a single x_{ij} only excludes one arc, requiring up to $O(n^2)$ branching decisions to resolve all fractional arcs.
- LP models are deep-copied via `Highs_passLp` for thread safety.
- Duplicate SECs are tracked via a hash set to avoid redundant cuts.
- Maximum branching depth is capped at $D_{\max} = 15$.
- Time limit $T_{\max} = 900\,\text{s}$ (15 minutes).

6 Computational Pipeline

The full pipeline consists of two stages:

Stage 1: Cost Surface Generation (Python).

1. Parse OMAP file → rasterise symbols onto UTM grid → ACR.
2. Load DTM, compute TRI, PC, SLO → normalise → HCR (eq. (6)).
3. Scale HCR via IQR ratio (eq. (7)), merge with ACR (eq. (8)).
4. Compute directional slopes from DTM.
5. Run anisotropic Dijkstra from each node with Minetti factors.
6. Export asymmetric cost matrix, scores, and budget as JSON.

Stage 2: Route Optimisation (C++).

1. Parse JSON input.
2. Run iterated SA → warm-start solution.
3. Build LP model with HiGHS, run B&C (algorithm 2).
4. Report solution, optimality status, and timing.

7 Experimental Setup

7.1 Study Areas

Table 2: Terrain characteristics of the two study areas.

	Torremocha	**La Muela**
Grid size	1056×1463	1376×1622
Resolution	2.0 m	2.0 m
Area	2.1×2.9 km	2.8×3.2 km
Elevation range	447–501 m	1164–1628 m
Relief	55 m	464 m
Median slope	3.2°	15.9°
Max slope	58.9°	81.2°
CRS	EPSG:25829	EPSG:25830
IQR scaling c	0.87	2.18
HCR median increase	+14.8%	+55.1%
Cost asymmetry	16.1%	50.0%

Torremocha is a gently undulating terrain in Extremadura, Spain, with moderate forest cover and extensive rock features. La Muela is a mountainous area with steep slopes, cliffs, and significant elevation variation. Additional study areas are being coordinated with the regional orienteering federation to broaden the validation set.

7.2 Control Distributions

Seven control-point distributions are generated for each terrain to test solver robustness:

1. **Standard**: grid-zone uniform spread (40 controls).
2. **Clustered**: 5 hotspot clusters (35 controls).
3. **Ring**: concentric rings from depot (30 controls).
4. **Path-biased**: 60% near paths, 40% far (35 controls).
5. **Elevation-biased**: weighted toward high elevation (35 controls).
6. **Sparse-far**: few controls, all far from depot (25 controls).
7. **Mixed-density**: dense near depot, sparse far (40 controls).

Point values (10–50) are assigned based on terrain difficulty at each control location.

7.3 Parameters

Race duration: 1 hour. Fatigue rate: $\lambda = 0.2$. Base speed: 2.5 m/s. Dijkstra downsampling: $s = 8$. B&C time limit: 900 s. SA iterations: auto-calibrated per instance. These values reflect the typical duration of a middle-distance orienteering race.

8 Results

8.1 Torremocha Results

Table 3: Torremocha results: SA vs. B&C(SA) across seven distributions (HCR-enhanced cost matrix, IQR scaling $c = 0.87$). The "visited" column indicates the number of controls visited in the solution (out of the total available).

Distribution	**Total**	**SA**			**B&C(SA)**				
	ctrls	pts	visited	time	pts	visited	time	optimal?	gap
Standard	40	930	26	0.3 s	930	26	900.4 s	no (limit)	15.3%
Clustered	35	510	29	0.5 s	510	29	136.8 s	**YES**	0.0%
Ring	30	650	26	0.4 s	650	26	179.9 s	**YES**	0.0%
Path-biased	35	630	30	0.5 s	630	30	423.6 s	**YES**	0.0%
Elev-biased	35	380	30	0.5 s	380	30	882.8 s	**YES**	0.0%
Sparse-far	25	660	19	0.3 s	660	19	169.1 s	**YES**	0.0%
Mixed-density	40	680	32	1.1 s	680	32	901.9 s	no (limit)	8.6%

On all seven instances, the B&C incumbent matches the SA warm-start solution. The B&C proves global optimality on 5 of 7 instances within the 15-minute time limit, certifying that no higher-scoring feasible route exists (see proposition 1 below). The two unproven instances (Standard and Mixed-density) have remaining LP relaxation gaps of 15.3% and 8.6% respectively, providing upper bounds on the possible improvement over the SA solution.

8.2 La Muela Results

Table 4: La Muela results: SA vs. B&C(SA) across seven distributions (HCR-enhanced cost matrix, IQR scaling $c = 2.18$, asymmetry 50.0%).

Distribution	**Total**	**SA**			**B&C(SA)**				
	ctrls	pts	visited	time	pts	visited	time	optimal?	gap
Standard	31	610	14	0.6 s	610	14	439.3 s	**YES**	0.0%
Clustered	35	800	24	1.4 s	800	24	43.3 s	**YES**	0.0%
Ring	30	470	21	1.0 s	470	21	790.4 s	**YES**	0.0%
Path-biased	35	570	17	1.3 s	570	17	109.3 s	**YES**	0.0%
Elev-biased	35	730	22	1.1 s	730	22	900.1 s	no (limit)	17.0%
Sparse-far	25	600	13	0.7 s	600	13	135.6 s	**YES**	0.0%
Mixed-density	40	680	25	1.4 s	680	25	900.3 s	no (limit)	19.5%

La Muela exhibits the same pattern as Torremocha: SA matches B&C on all seven instances, with 5 of 7 proven optimal. Despite the significantly harder terrain (464 m relief, 50% cost asymmetry, IQR scaling $c = 2.18$), the B&C closes the optimality gap on most instances. The two unproven instances (Elev-biased and Mixed-density) have LP relaxation gaps of 17.0% and 19.5%, larger than the Torremocha gaps, reflecting the increased difficulty of the asymmetric LP relaxation on mountainous terrain. Notably, the SA runtime is slightly higher on La Muela (0.6–1.4 s vs. 0.3–1.1 s on Torremocha), reflecting the steeper terrain which produces larger traversal costs and deeper feasibility evaluations per move.

Across both terrains (14 instances total), SA finds the B&C-verified optimum on all 10 proven instances, providing strong empirical evidence for the effectiveness of the tuned SA on real-terrain

OP instances.

8.3 Synthetic Benchmark Instances

To the best of our knowledge, no standard benchmark exists for the asymmetric OP with fatigue. We generate a parameterized benchmark suite by varying four dimensions independently from a baseline configuration ($n = 40$, $\alpha = 0.25$, $\lambda = 0.2$, medium budget):

- Node count $n \in \{20, 30, 40, 50, 75, 100\}$
- Asymmetry parameter $\alpha \in \{0.0, 0.1, 0.25, 0.5\}$, controlling directional cost perturbation
- Fatigue rate $\lambda \in \{0.0, 0.1, 0.2, 0.3\}$
- Budget tightness: loose (∼70%), medium (∼50%), tight (∼30%), defined as a fraction of the nearest-neighbour tour cost

Additionally, four extreme-case instances test boundary conditions (e.g., symmetric with no fatigue, highly asymmetric with tight budget). Nodes are placed uniformly at random in a 1000×1000 m domain with the depot at the centre. Asymmetric costs are generated by perturbing Euclidean distances with a directional slope factor proportional to α, plus 10% random terrain noise. Scores are correlated with distance from the depot (10–50 points, rounded to multiples of 10). All instances and the solver are released as part of the benchmark suite.

8.4 Benchmark Results

Table 5: Benchmark results: effect of instance size (baseline: $\alpha = 0.25$, $\lambda = 0.2$, medium budget).

n	SA pts	visited	SA time	B&C time	optimal?	gap
20	350	12	0.5 s	4.8 s	**YES**	0.0%
30	540	18	1.0 s	123.7 s	**YES**	0.0%
40	660	26	1.5 s	900.2 s	no (limit)	17.8%
50	940	31	1.9 s	901.3 s	no (limit)	16.5%
75	1360	43	2.3 s	900.7 s	no (limit)	19.9%
100	1830	60	1.7 s	903.2 s	no (limit)	21.0%

Table 6: Benchmark results: effect of asymmetry and fatigue ($n = 40$, medium budget).

Parameter	Value	SA pts	B&C time	optimal?	gap
Asymmetry α	0.00	690	900.3 s	no (limit)	16.8%
	0.10	730	900.2 s	no (limit)	14.1%
	0.25	670	900.1 s	no (limit)	20.5%
	0.50	620	901.1 s	no (limit)	24.6%
Fatigue λ	0.00	790	900.4 s	no (limit)	6.6%
	0.10	730	900.3 s	no (limit)	7.7%
	0.20	730	901.1 s	no (limit)	15.5%
	0.30	750	900.5 s	no (limit)	24.9%

The benchmark reveals several patterns. First, the B&C proves optimality for $n \leq 30$ within the time limit but struggles at $n \geq 40$, with gaps growing from 17.8% to 21.0% as instance size increases (table 5). Second, higher asymmetry widens the LP gap (16.8% at $\alpha = 0$ to 24.6% at $\alpha = 0.5$), confirming that asymmetric costs make the LP relaxation looser (table 6). Third, fatigue has a pronounced effect on gap quality: without fatigue ($\lambda = 0$) the gap is only 6.6%, rising to 24.9% at $\lambda = 0.3$, indicating that the McCormick linearisation introduces significant

relaxation looseness at high fatigue rates. The hardest instance (asymmetric_hard: $n = 100$, $\alpha = 0.5$, $\lambda = 0.3$, tight budget) has a 42.8% gap, while the easiest (symmetric_easy: $n = 20$, $\alpha = 0$, $\lambda = 0$) solves in under 1 second.

Proposition 1 (Optimality certificate)**.** *Let z^*_{SA} denote the objective value of the SA incumbent solution, and let $\mathcal{T}$ denote the Branch-and-Cut search tree. For each node $\nu \in \mathcal{T}$, let $\bar{z}(\nu)$ denote the optimal value of the LP relaxation at ν (an upper bound on the best integer-feasible solution in the subtree rooted at ν). If the B&C algorithm terminates with an empty node stack (i.e., all nodes have been either explored or pruned), then z^*_{SA} is the globally optimal value of the AOPF.*

Proof. The B&C algorithm partitions the feasible region of the AOPF into a binary search tree $\mathcal{T}$ by branching on fractional y_i variables. At each node ν, the LP relaxation (with dynamically added subtour elimination and connectivity cuts) yields an upper bound $\bar{z}(\nu) \geq z^*(\nu)$, where $z^*(\nu)$ is the optimal integer solution in the subtree rooted at ν. This bound is valid because:

1. The LP relaxation is a relaxation of the integer program (all integer constraints are dropped, and the feasible set is enlarged).
2. The subtour elimination cuts (eq. (30)) and connectivity cuts (eqs. (31) and (32)) are valid inequalities—they do not exclude any integer-feasible solution.

A node ν is pruned if $\bar{z}(\nu) \leq z^*_{\text{SA}} + \varepsilon$ (with $\varepsilon = 10^{-6}$), meaning no integer solution in its subtree can improve upon the incumbent. A node is also pruned if its LP relaxation is infeasible, or if an integer-feasible solution is found and used to update the incumbent.

When the algorithm terminates with an empty stack, every leaf of $\mathcal{T}$ has been resolved by one of:

- *Bound pruning*: $\bar{z}(\nu) \leq z^*_{\text{SA}} + \varepsilon$, certifying that no better solution exists in this subtree.
- *Infeasibility*: the LP relaxation at ν has no feasible solution.
- *Integer solution*: an integer-feasible solution was found and, if superior, became the new incumbent.

Since the branching is exhaustive (every fractional variable is eventually fixed to 0 or 1) and the union of all subtrees covers the entire feasible region of the AOPF, the incumbent at termination is a globally optimal solution. In particular, if the incumbent remains the SA solution z^*_{SA}, then no integer-feasible solution with objective value strictly greater than z^*_{SA} exists. □

9 Discussion

SA effectiveness. The SA heuristic matches the proven optimum on all 10 certified instances across both terrains, suggesting it is highly effective for OP instances of this size ($n \leq 40$). This is consistent with the literature on metaheuristics for the OP [Vansteenwegen et al., 2011].

Cost asymmetry. The 50% asymmetry on La Muela demonstrates that symmetric OP formulations would be inadequate for mountainous terrain. The Minetti model captures this directionality naturally through the metabolic cost polynomial.

Fatigue linearisation. The McCormick envelope approximates the bilinear fatigue term $t_i \cdot x_{ij}$, with tightened upper bounds $\bar{w}_{ij}$ that exploit the OP structure (must return to depot). However, the benchmark results confirm that the McCormick envelope is the primary source of LP relaxation looseness: the gap increases from 6.6% without fatigue ($\lambda = 0$) to 24.9% at $\lambda = 0.3$ (table 6), indicating that the linear outer approximation becomes increasingly loose at higher fatigue rates. Tighter bilinear relaxations (e.g., piecewise McCormick or spatial branching on the t_i variables) could substantially improve B&C performance on high-fatigue instances.

Computational complexity and runtime. The overall framework has three computational bottlenecks, each with distinct scaling behaviour.

Cost matrix construction. For n control points on a grid of $H \times W$ cells downsampled by factor s, the anisotropic Dijkstra runs n times on a grid of $(H/s)(W/s)$ cells. Each Dijkstra has complexity $O\big(\frac{HW}{s^2} \log \frac{HW}{s^2}\big)$, giving a total cost matrix construction time of $O\big(\frac{nHW}{s^2} \log \frac{HW}{s^2}\big)$. In practice, with Numba JIT compilation and $s = 8$, the cost matrix for 41 nodes on a 1056×1463 grid computes in under 4 seconds.

Simulated Annealing. Each SA iteration evaluates a neighbourhood move in $O(k)$ time, where k is the current route length. With N_{iter} iterations and N_{restart} restarts, the total SA time is $O(N_{\text{restart}} \cdot N_{\text{iter}} \cdot k)$. Since $k \leq n$ and both N_{iter} and N_{restart} are bounded by constants dependent on n, the SA runs in $O(n^2)$ per restart in the worst case. Empirically, SA completes in under 1 second for all tested instances ($n \leq 41$).

Branch-and-Cut. The B&C has worst-case exponential complexity in n, as the number of branching nodes can grow as $O(2^n)$. The LP relaxation at each node has $O(n^2)$ variables and $O(n^2)$ constraints (before cuts), solvable in polynomial time by the simplex method. However, the number of subtour elimination cuts can be exponential in $|V|$. In our experiments, the B&C runtime varies dramatically with instance structure: instances with fewer nodes or simpler topology (e.g., Clustered with 24 nodes: 14.1 s) are solved orders of magnitude faster than dense instances (e.g., Standard with 31 nodes: 900 s, hitting the time limit). This exponential sensitivity to instance structure is characteristic of branch-and-cut methods for routing problems and motivates the SA warm start, which provides a strong incumbent that enables aggressive pruning early in the search.

Limitations. The framework has not been validated against GPS tracks from real orienteers. The fatigue model is linear, whereas real fatigue may follow a more complex curve (e.g., exponential degradation or threshold effects at high intensity). The B&C does not scale well beyond $\sim$50 nodes due to the exponential growth of the branching tree; for larger instances, the SA solution should be treated as a high-quality heuristic without an optimality guarantee. The Dijkstra downsampling ($s = 8$) introduces path approximation error that has not been formally quantified, though empirical comparison with $s = 4$ suggests the effect on the cost matrix is small ($< 2\%$ median deviation).

10 Conclusion

We presented an integrated framework for optimal route planning in orienteering, combining GIS-based cost surface generation with exact combinatorial optimisation. The key methodological contributions are: (i) the combination of ACR, HCR, and Minetti metabolic cost into a single asymmetric cost model; (ii) IQR-based raster fusion that adapts to terrain characteristics; (iii) a Branch-and-Cut formulation with McCormick linearisation of cumulative fatigue; and (iv) computational evidence that Simulated Annealing finds optimal solutions on real-terrain instances.

Future work includes validation against real GPS data, extension to multi-day events, and integration of weather and time-of-day effects on terrain traversability.

Reproducibility. The source code, benchmark instances (14 JSON files with asymmetric cost matrices and proven optimal solutions), and supplementary data are publicly available at `https://github.com/PabloAlmendralejo/Orienteering-Problem/tree/main`. To the best of our knowledge, no benchmark instances exist for the asymmetric OP with terrain-based costs and fatigue; we release ours to facilitate future comparison.

References

T. Tsiligirides. Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35(9):797–809, 1984.

A. E. Minetti, C. Moia, G. S. Roi, D. Susta, and G. Ferretti. Energy cost of walking and running at extreme uphill and downhill slopes. *Journal of Applied Physiology*, 93(3):1039–1046, 2002.

G. Albert and Z. Sárközy. Route planning on orienteering maps with least-cost path analysis. *Proceedings of the ICA*, 4:4, 2021. doi:10.5194/ica-proc-4-4-2021.

S. J. Riley, S. D. DeGloria, and R. Elliot. A terrain ruggedness index that quantifies topographic heterogeneity. *Intermountain Journal of Sciences*, 5(1–4):23–27, 1999.

L. W. Zevenbergen and C. R. Thorne. Quantitative analysis of land surface topography. *Earth Surface Processes and Landforms*, 12(1):47–56, 1987.

Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1–10, 2011.

M. Fischetti, J. J. Salazar González, and P. Toth. Solving the orienteering problem through branch-and-cut. *INFORMS Journal on Computing*, 10(2):133–148, 1998.

M. Gendreau, G. Laporte, and F. Semet. A branch-and-cut algorithm for the undirected selective travelling salesman problem. *Networks*, 32(4):263–273, 1998.

A. Gunawan, H. C. Lau, and P. Vansteenwegen. Orienteering problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research*, 255(2):315–332, 2016.

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

M. F. Tasgetiren. A genetic algorithm with an adaptive penalty function for the orienteering problem. *Journal of the Operational Research Society*, 53(3):263–270, 2002.

L. Ke, C. Archetti, and Z. Feng. Ants can solve the team orienteering problem. *Computers & Industrial Engineering*, 54(3):648–665, 2008.

P. Vansteenwegen, W. Souffriau, G. Vanden Berghe, and D. Van Oudheusden. Iterated local search for the team orienteering problem with time windows. *Computers & Operations Research*, 36(12):3281–3290, 2009.

W. Tobler. Three presentations on geographical analysis and modeling. Technical Report 93-1, National Center for Geographic Information and Analysis, 1993.

W. G. Rees. Least-cost paths in mountainous terrain. *Computers & Geosciences*, 30(3):203–209, 2004.